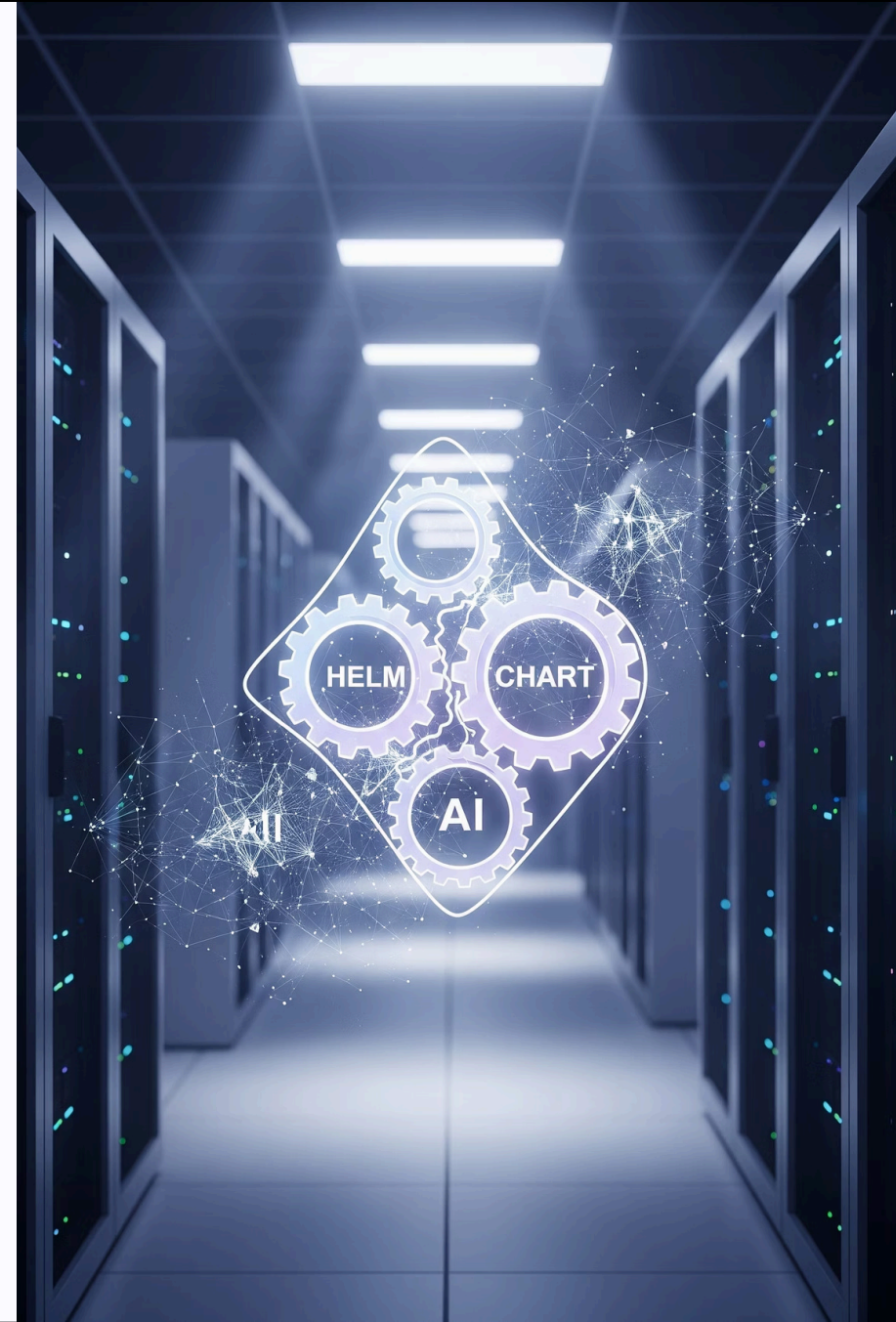


Helm Advanced Templates for AI Apps

A practical guide for DevOps engineers deploying machine learning workloads on Kubernetes

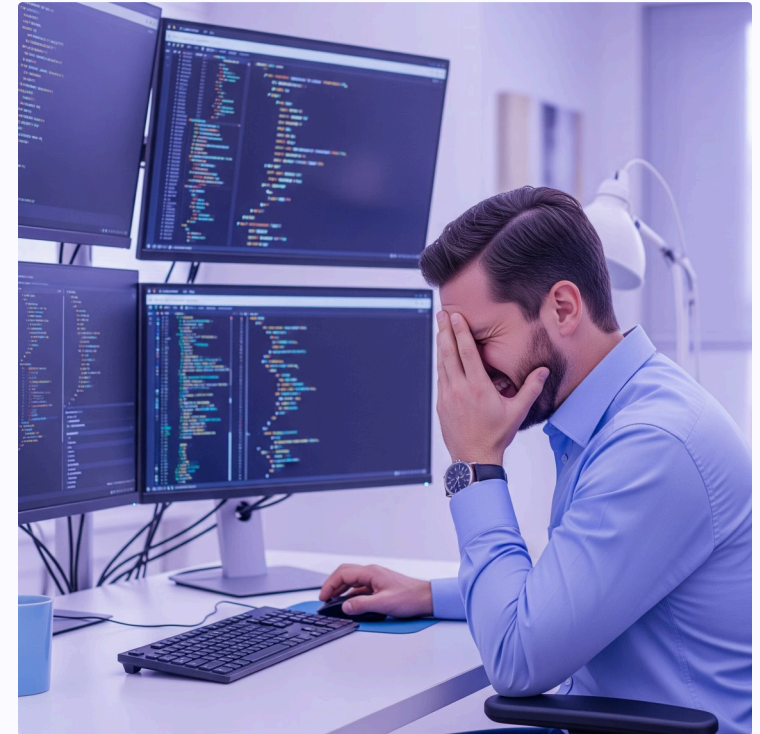


Why Helm for AI Apps?

AI applications demand **complex Kubernetes manifests** that include numerous interdependent resources:

- Stateful model servers with GPU requirements
- Inference endpoints with autoscaling configurations
- Model storage with versioning
- Specialized monitoring and logging

Writing these configurations by hand becomes **repetitive, error-prone, and unmanageable** at scale.



Helm's templating engine enables **parameterized, reusable deployments** that are essential for scaling AI infrastructure teams efficiently.

Helm Recap

1

Chart Structure

Packaged K8s resources with a standardized directory layout containing templates, values, and metadata

2

Values System

External configuration via values.yaml with override capabilities at install time

3

Templating Engine

Go templates to substitute variables and apply logic to YAML generation

4

Release Management

Built-in support for upgrades, rollbacks, and versioning

Helm makes AI deployments **repeatable and manageable** across environments and team members.

Advanced Templating Features



Variables & Functions

`{{ .Values.image }}` and built-in functions like `quote`, `default`, and `toYaml`



Conditionals

`{{- if .Values.feature.enabled }}...{{- end }}` for optional components



Loops

`{{- range .Values.models }}` to generate multiple similar resources



Named Templates

`{{- include "chart.labels" . }}` for reusable template snippets

These features transform Helm templates into powerful [infrastructure-as-code solutions](#) with the flexibility AI applications demand.

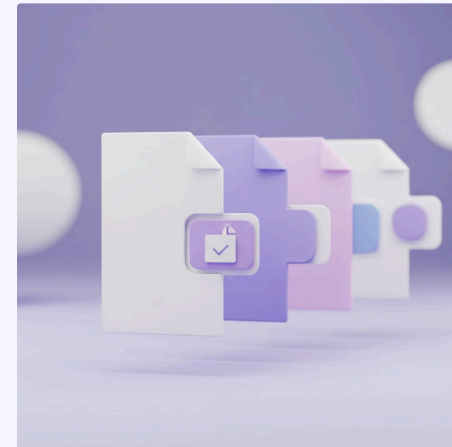
Example – Parameterized Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.app.name }}
spec:
  replicas: {{ .Values.app.replicas }}
  template:
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        resources:
          limits:
            memory: {{ .Values.resources.memory }}
            cpu: {{ .Values.resources.cpu }}
```

Template Advantages

- Dynamic app name from values
- Configurable replica count
- Flexible image selection
- Resource limits as parameters
- Single source of truth

All without hard-coding values in YAML!



Values.yaml Example

```
app:
  name: ai-inference
  replicas: 3

image:
  repository: myrepo/ai-app
  tag: latest

resources:
  memory: "4Gi"
  cpu: "2"

monitoring:
  enabled: true
```

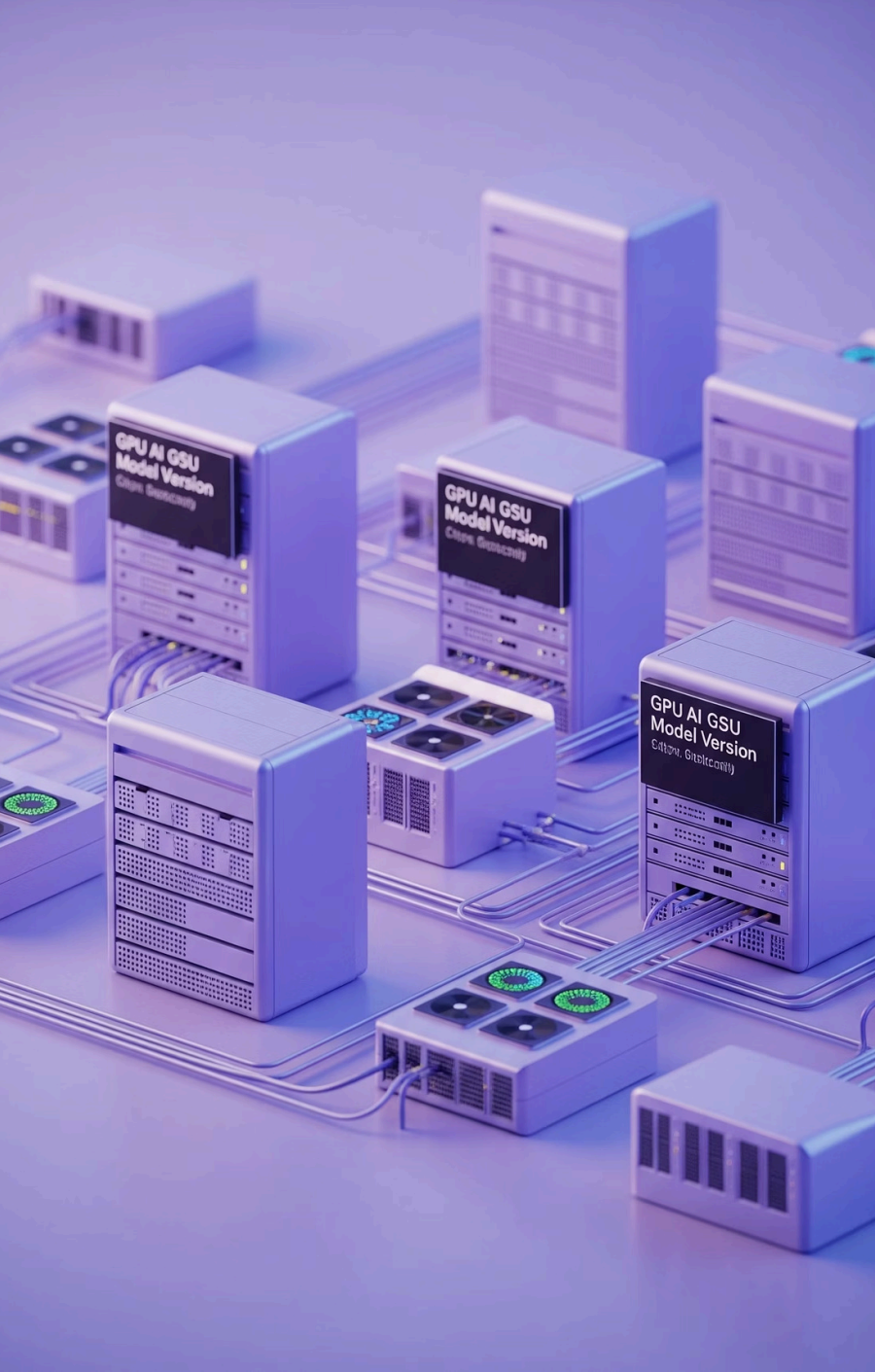
Override at Install Time

```
helm install ai-app ./chart \
  --set app.replicas=5 \
  --set image.tag=v2.0.3 \
  --set resources.memory=8Gi
```

Values can be overridden via:

- Command-line flags
- Additional values files
- Environment-specific values files

This approach enables [consistent yet flexible deployments](#) across development, staging, and production environments.



Helm for AI-Specific Needs

Dynamic GPU Configuration

Configure GPU requests based on model size, with fallbacks to CPU when GPUs aren't available

Multi-Model Deployments

Generate services, deployments, and configs for multiple models from a single values list

Model Versioning

Parameterize model versions for easy updates (model:v1, model:v2) and A/B testing

Environment Adaptation

Adjust monitoring, logging, and resource requests based on deployment environment

Example – GPU Request Template

```
{{- if .Values.gpu.enabled }}
resources:
  limits:
    nvidia.com/gpu: {{ .Values.gpu.count | default 1 }}
{{- else }}
resources:
  limits:
    cpu: {{ .Values.cpu.limit }}
    memory: {{ .Values.memory.limit }}
{{- end }}
```

Values.yaml:

```
gpu:
  enabled: true
  count: 2

# Fallback resources if GPUs disabled
cpu:
  limit: "4"
memory:
  limit: "16Gi"
```

Flexibility Benefits

- Conditional GPU allocation
- Graceful fallback to CPU-only
- Environment-specific configs:
 - Dev: no GPUs
 - Staging: 1 GPU
 - Prod: 2+ GPUs



Benefits of Helm Advanced Templates

Infrastructure as Code

Versioned, reproducible deployments with Git-tracked templates and values

DRY Principle

No duplicate YAML across environments or similar deployments, reducing maintenance burden

Deployment Consistency

Standardized configurations across dev, staging, and production environments

Lifecycle Management

Easy upgrades, rollbacks, and audit trail for all deployed AI applications

Team Enablement

Self-service deployments for data scientists with guardrails built into templates

Multi-Model Support

Deploy and manage multiple AI models with consistent configuration patterns



Challenges & Best Practices

Common Challenges

- Complex charts can be difficult to debug
- Templating expertise required for maintenance
- Excessive templating creates unreadable YAML
- Security risks with secrets in values.yaml
- May introduce overhead for simple projects

Best Practices

- Use named templates for repeated patterns
- Document values with clear comments
- Include sensible defaults
- Validate values with helpers
- Use external secrets management
- Create a test suite for templates

Key Takeaways

Helm is the de facto package manager for Kubernetes AI applications

It provides essential tooling for deploying complex, multi-component AI systems

Advanced templating enables AI-specific configuration needs

Parameterize GPU allocation, model versions, replicas, and services for maximum flexibility

Template-driven deployment is critical for enterprise AI

Boosts scalability, consistency, and manageability across teams and environments

Next steps: Explore our Helm chart repository with AI-specific templates and examples at github.com/your-org/ai-helm-charts

